



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student/s Name	1. ARIANNE BEVERLY MARQUEZ 2. LESTER ANTHONY RESPICIO
Section:	AN41
Professor:	MS. CRISOLA TAN
Date:	10/03/26

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF file**.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Chosen Object: Medical Symptoms of Heart Attack

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?

→ Rule-Based Reasoning (RBR) is typically easier to implement initially for small, well-defined scopes. Setting up the RBR engine just required writing straightforward if-then conditional functions and a simple loop to check conditions against the facts. Unlike Case-Based Reasoning (CBR), which required significantly more structural overhead, including initializing a dictionary of past

cases, establishing a mathematical similarity function to compare problem keys, and setting up the logic to handle the Revise and Retain phases of the CBR cycle.

2. Which is more flexible in solving unstructured problems?

→ CBR is far more flexible. This is because CBR handles ambiguity by calculating a similarity score to find the "closest match," allowing the system to suggest a solution even if the exact combination of variables hasn't been explicitly programmed before. RBR operates on rigid logic; if a problem's variables don't perfectly trigger the exact conditions of a predefined rule, the RBR engine returns nothing.

3. Which is better for situations with continuously changing data?

→ CBR is significantly better for dynamic environments. It is inherently designed to learn over time, for example when a new case is processed, the system can revise the suggested solution and retain it in the case base (just like the UI retention feature in our code). This expands the system's knowledge immediately without altering the core script. Compared with RBR, adapting to new data requires a developer to manually write, test, and deploy new rules.

4. Examples of real-world applications for both.

→ **Rule-Based Reasoning (RBR): Banking fraud detection**

- ❖ In this scenario, a system relies on `if-then` parameters to monitor transactions in real time. For example, if a customer's credit card was suddenly used for a ₱ 50,000 purchase in a foreign country, the RBR engine evaluates the data against its programmed security threshold wherein **IF** the transaction location does not match the home country **AND** the amounts exceeds ₱20,000, **THEN** instantly freeze the account and flag it for review.

The system does not analyze historical fraud trends, it just simply executes the hardcoded protocol the moment the conditions are met.

→ **Case-Based Reasoning (CBR): Customer support helpdesk systems**

- ❖ An example for CBR is in a telecommunications customer support helpdesk, a representative might receive a ticket regarding a disputed fiber connection paired with a rare router error code. Instead of forcing the customer representative through a firm diagnostic tree that might lack this exact combination, the CBR system analyzes the new ticket's variables and searches thousands of historically resolved accounts. It retrieves a past case with a highly similar error pattern, in order to suggest a successful troubleshooting steps from that historical ticket, and retains any new workarounds the representative applies to expand its knowledge base for future connectivity issues.

Your Code - Laboratory Tasks

```
import tkinter as tk
import os

# PART 1: Knowledge Representation (KR) Setup

# Task 1A: Create a Knowledge Base
facts = {
    "chest_pain_severity": 8,
    "shortness_of_breath": True,
    "dizziness": True,
    "left_arm_numbness": True,
    "systolic_bp": 165
}
```

```
# PART 2: Implement Rule-Based Reasoning (RBR)
```

```
# Task 2A: Create at least 3 IF THEN rules
```

```
def rule_severe_pain(f):  
    if f.get("chest_pain_severity", 0) >= 7 and  
f.get("left_arm_numbness") == True:  
        return "Administer aspirin and call emergency services  
immediately."  
    return None
```

```
def rule_high_bp(f):  
    if f.get("systolic_bp", 0) > 140 and f.get("dizziness") ==  
True:  
        return "Monitor blood pressure closely and keep the  
patient seated."  
    return None
```

```
def rule_breathing(f):  
    if f.get("shortness_of_breath") == True:  
        return "Provide supplemental oxygen."  
    return None
```

```
# Task 2B: Implement an Inference Engine
```

```
def inference_engine(current_facts, rule_set):  
    inferred_actions = []  
    for rule in rule_set:  
        action = rule(current_facts)  
        if action:  
            inferred_actions.append(action)  
    return inferred_actions
```

```
# PART 3: Implement Case-Based Reasoning (CBR)
```

```
# Task 3A: Build a Case Base
```

```
case_base = [  
    {  
        "problem": {"chest_pain_severity": 2,  
"shortness_of_breath": False, "left_arm_numbness": False},  
        "solution": "Recommend rest and antacids. Monitor for 30  
minutes."  
    },  
    {
```

```

        "problem": {"chest_pain_severity": 9,
"shortness_of_breath": True, "left_arm_numbness": True},
        "solution": "Code Red: Suspected Myocardial Infarction.
Dispatch ambulance immediately."
    },
    {
        "problem": {"chest_pain_severity": 0,
"shortness_of_breath": False, "left_arm_numbness": True},
        "solution": "Assess for pinched nerve or localized
musculoskeletal strain."
    }
]

```

```

def get_similarity(past_problem, new_problem):
    matches = 0
    for key in new_problem:
        if new_problem.get(key) == past_problem.get(key):
            matches += 1
    return matches

```

```

# CUSTOM MEDICAL GUI INTERFACE

```

```

class MedicalExpertSystemUI:
    def __init__(self, root):
        self.root = root
        self.root.withdraw()

    def abort_program(self):
        """Kills the entire program instantly if the 'X' button
is clicked."""
        print("System terminated by user.")
        os._exit(0) # Forces an immediate operating system level
exit

```

```

    def input_new_case(self):
        win = tk.Toplevel(self.root)
        win.title("🚑 New Patient Entry")
        win.geometry("450x350")
        win.configure(bg="#FFFFFF")

        win.protocol("WM_DELETE_WINDOW", self.abort_program)

```



```
        tk.Label(win, text="Input Patient Symptoms 🩺",
font=("Helvetica", 16, "bold"), fg="#D32F2F",
bg="#FFFFFF").pack(pady=15)
```

```
        tk.Label(win, text="Chest Pain Severity (0-10):",
font=("Helvetica", 10, "bold"), fg="#1976D2",
bg="#FFFFFF").pack(pady=(10, 0))
```

```
        pain_var = tk.IntVar(value=0)
        tk.Scale(win, from_=0, to=10, orient="horizontal",
variable=pain_var, bg="#FFFFFF", highlightthickness=0,
length=200).pack()
```

```
        breath_var = tk.BooleanVar()
        tk.Checkbutton(win, text="Patient has Shortness of
Breath", variable=breath_var, font=("Helvetica", 10),
bg="#FFFFFF", fg="#333333", selectcolor="#FFFFFF").pack(pady=5)
```

```
        numb_var = tk.BooleanVar()
        tk.Checkbutton(win, text="Patient has Left Arm
Numbness", variable=numb_var, font=("Helvetica", 10),
bg="#FFFFFF", fg="#333333", selectcolor="#FFFFFF").pack(pady=5)
```

```
        self.new_case_data = None
```

```
        def submit_case():
            self.new_case_data = {
                "chest_pain_severity": pain_var.get(),
                "shortness_of_breath": breath_var.get(),
                "left_arm_numbness": numb_var.get()
            }
            win.destroy()
```

```
        tk.Button(win, text="Analyze Case", bg="#D32F2F",
fg="white", font=("Helvetica", 10, "bold"),
command=submit_case).pack(pady=20)
```

```
        self.root.wait_window(win)
        return self.new_case_data
```

```
        def show_results(self, rbr_text, cbr_text):
            win = tk.Toplevel(self.root)
            win.title("💊 Medical AI Diagnostics")
            win.geometry("550x450")
            win.configure(bg="#FFFFFF")
```

```

win.protocol("WM_DELETE_WINDOW", self.abort_program)

tk.Label(win, text="AI Diagnostic Results 🧠",
font=("Helvetica", 18, "bold"), fg="#D32F2F",
bg="#FFFFFF").pack(pady=15)

tk.Label(win, text="Rule-Based Reasoning (RBR):",
font=("Helvetica", 11, "bold"), fg="#1976D2",
bg="#FFFFFF").pack(anchor="w", padx=20)
rbr_box = tk.Text(win, font=("Helvetica", 10),
fg="#333333", bg="#FFFFFF", wrap="word", height=4, bd=0,
highlightthickness=0)
rbr_box.insert("1.0", rbr_text)
rbr_box.config(state="disabled")
rbr_box.pack(fill="x", padx=30, pady=5)

tk.Label(win, text="Case-Based Reasoning (CBR):",
font=("Helvetica", 11, "bold"), fg="#1976D2",
bg="#FFFFFF").pack(anchor="w", padx=20, pady=(15, 0))
cbr_box = tk.Text(win, font=("Helvetica", 10),
fg="#333333", bg="#FFFFFF", wrap="word", height=4, bd=0,
highlightthickness=0)
cbr_box.insert("1.0", cbr_text)
cbr_box.config(state="disabled")
cbr_box.pack(fill="x", padx=30, pady=5)

btn = tk.Button(win, text="Acknowledge & Continue",
bg="#1976D2", fg="white", font=("Helvetica", 10, "bold"),
command=win.destroy)
btn.pack(pady=25)

self.root.wait_window(win)

def ask_revision(self, suggested_solution):
win = tk.Toplevel(self.root)
win.title("🔧 Revise Solution Step")
win.geometry("550x250")
win.configure(bg="#FFFFFF")

win.protocol("WM_DELETE_WINDOW", self.abort_program)

self.revised_solution = None

```

```

        tk.Label(win, text="Revise CBR Solution? 🧐",
font=("Helvetica", 14, "bold"), fg="#D32F2F",
bg="#FFFFFF").pack(pady=15)

```

```

        entry = tk.Entry(win, width=70, font=("Helvetica", 10))
        entry.insert(0, suggested_solution)
        entry.pack(pady=15, padx=20)

```

```

        def save():
            self.revised_solution = entry.get()
            win.destroy()

```

```

        def keep():
            self.revised_solution = suggested_solution
            win.destroy()

```

```

        tk.Button(win, text="Save Revision", bg="#D32F2F",
fg="white", font=("Helvetica", 10, "bold"),
command=save).pack(side="left", padx=60, pady=20)
        tk.Button(win, text="Keep Original", bg="#1976D2",
fg="white", font=("Helvetica", 10, "bold"),
command=keep).pack(side="right", padx=60, pady=20)

```

```

        self.root.wait_window(win)
        return self.revised_solution

```

```

        def show_retained(self, count):
            win = tk.Toplevel(self.root)
            win.title("📌 Case Base Updated")
            win.geometry("350x180")
            win.configure(bg="#FFFFFF")

```

```

            win.protocol("WM_DELETE_WINDOW", self.abort_program)

```

```

            tk.Label(win, text="✅ Case Retained Successfully",
font=("Helvetica", 12, "bold"), fg="#1976D2",
bg="#FFFFFF").pack(pady=20)

```

```

            tk.Label(win, text=f"Total cases in Case Base now:
{count}", font=("Helvetica", 11), fg="#D32F2F",
bg="#FFFFFF").pack()

```

```

            tk.Button(win, text="View Reports", bg="#1976D2",
fg="white", font=("Helvetica", 10, "bold"),
command=win.destroy).pack(pady=20)

```

```

        self.root.wait_window(win)

    def show_reports(self, cases):
        win = tk.Toplevel(self.root)
        win.title("🚑 System Reports")
        win.geometry("600x400")
        win.configure(bg="#FFFFFF")

        win.protocol("WM_DELETE_WINDOW", win.destroy)

        tk.Label(win, text="Case Base Reports 🏠",
font=("Helvetica", 16, "bold"), fg="#D32F2F",
bg="#FFFFFF").pack(pady=15)

        tk.Button(win, text="Close System", bg="#D32F2F",
fg="white", font=("Helvetica", 10, "bold"),
command=win.destroy).pack(side="bottom", pady=15)

        frame = tk.Frame(win, bg="#FFFFFF")
        frame.pack(fill="both", expand=True, padx=20, pady=(0,
10))

        scrollbar = tk.Scrollbar(frame)
        scrollbar.pack(side="right", fill="y")

        text_area = tk.Text(frame, font=("Helvetica", 10),
bg="#F9F9F9", fg="#333333", wrap="word",
yscrollcommand=scrollbar.set, padx=15, pady=10)

        report_content = ""
        for i, case in enumerate(cases):
            report_content += f"CASE {i+1}:\n"
            report_content += f"Problem Variables:
{case['problem']}\n"
            report_content += f"Assigned Solution:
{case['solution']}\n"
            report_content += "-"*60 + "\n\n"

        text_area.insert("1.0", report_content)
        text_area.config(state="disabled")
        text_area.pack(side="left", fill="both", expand=True)
        scrollbar.config(command=text_area.yview)

        self.root.wait_window(win)

```

```

def run_expert_system():
    root = tk.Tk()
    ui = MedicalExpertSystemUI(root)

    rules_list = [rule_severe_pain, rule_high_bp,
rule_breathing]
    actions_to_take = inference_engine(facts, rules_list)
    rbr_output = "\n".join([f"• {action}" for action in
actions_to_take])

    new_patient_problem = ui.input_new_case()

    # Extra safety check
    if not new_patient_problem:
        os._exit(0)

    best_match = None
    highest_score = -1
    for case in case_base:
        score = get_similarity(case["problem"],
new_patient_problem)
        if score > highest_score:
            highest_score = score
            best_match = case

    suggested_solution = best_match["solution"]
    cbr_output = f"Most similar case found (Match Score:
{highest_score}).\nSuggested Solution: {suggested_solution}"

    ui.show_results(rbr_output, cbr_output)
    final_solution = ui.ask_revision(suggested_solution)

    new_case = {"problem": new_patient_problem, "solution":
final_solution}
    case_base.append(new_case)

    ui.show_retained(len(case_base))
    ui.show_reports(case_base)

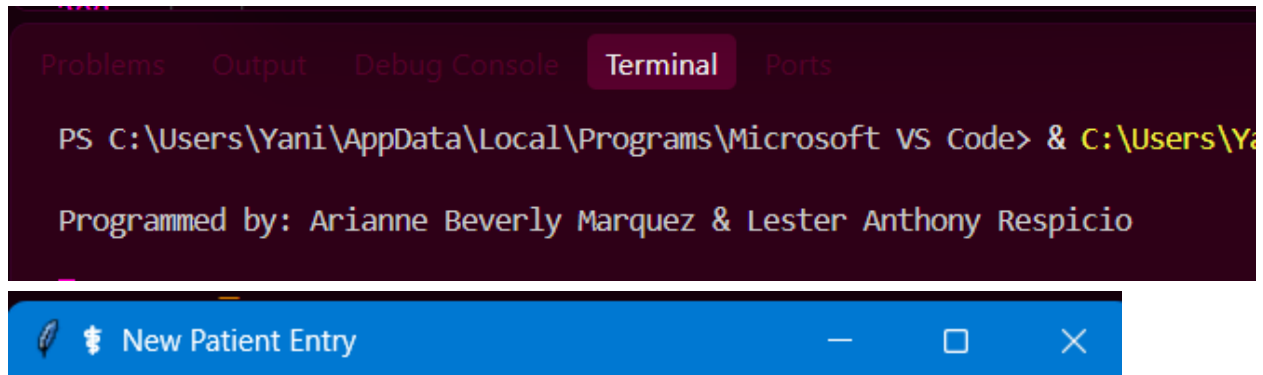
    root.destroy()

if __name__ == "__main__":

```

```
print("\nProgrammed by: Arianne Beverly Marquez & Lester  
Anthony Respicio\n")  
run_expert_system()
```

Program Screenshots:



Input Patient Symptoms 🏠

Chest Pain Severity (0-10):

10

☒ Patient has Shortness of Breath

☒ Patient has Left Arm Numbness

Analyze Case

AI Diagnostic Results

Rule-Based Reasoning (RBR):

- Administer aspirin and call emergency services immediately.
- Monitor blood pressure closely and keep the patient seated.
- Provide supplemental oxygen.

Case-Based Reasoning (CBR):

Most similar case found (Match Score: 2).

Suggested Solution: Code Red: Suspected Myocardial Infarction. Dispatch ambulance immediately.

Acknowledge & Continue

 Revise Solution Step—□×

Revise CBR Solution?

Code Red: Suspected Myocardial Infarction. Dispatch ambulance immediately.

Save Revision

Keep Original

 Case Base Updated—□×

☒ **Case Retained Successfully**

Total cases in Case Base now: 4

View Reports

Case Base Reports

CASE 1:

Problem Variables: {'chest_pain_severity': 2, 'shortness_of_breath': False, 'left_arm_numbness': False}

Assigned Solution: Recommend rest and antacids. Monitor for 30 minutes.

CASE 2:

Problem Variables: {'chest_pain_severity': 9, 'shortness_of_breath': True, 'left_arm_numbness': True}

Assigned Solution: Code Red: Suspected Myocardial Infarction. Dispatch ambulance immediately.

CASE 3:

Problem Variables: {'chest_pain_severity': 0, 'shortness_of_breath': False, 'left_arm_numbness': True}

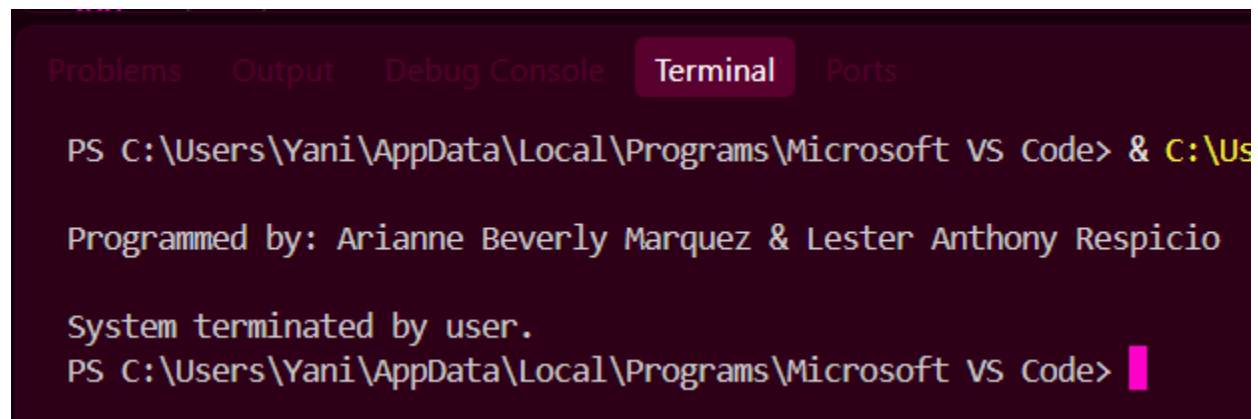
Assigned Solution: Assess for pinched nerve or localized musculoskeletal strain.

CASE 4:

Problem Variables: {'chest_pain_severity': 10, 'shortness_of_breath': True, 'left_arm_numbness': True}

Assigned Solution: Code Red: Suspected Myocardial Infarction. Dispatch ambulance immediately.

Close System



The image shows a terminal window from the Visual Studio Code interface. The title bar at the top reads "Visual Studio Code". Below the title bar, there are tabs for "Problems", "Output", "Debug Console", "Terminal", and "Ports". The "Terminal" tab is currently selected. The terminal content shows a command prompt "PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code>" followed by a command "& C:\Us" (partially visible). The output of the command is "Programmed by: Arianne Beverly Marquez & Lester Anthony Respicio". Below this, it says "System terminated by user." and then the prompt "PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code>" appears again with a red cursor.

```
Visual Studio Code
Problems Output Debug Console Terminal Ports
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C:\Us
Programmed by: Arianne Beverly Marquez & Lester Anthony Respicio
System terminated by user.
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code>
```